

문서를 LLM 위키로 바꿨다

왜 바꿨고, 무엇이 달라졌으며, 4주 뒤에도 그 판단이 유효한가

기준일 2026년 8월 25일 · 개편은 2026년 7월 31일 (4주 경과)

규모 위키 노드 163개 (결정 91 · 표 40 · 색인/규약 13 · 운영 10 · 원본 9) · 링크 663개

핵심 수치 결정 하나를 찾는 비용 **33,792 → 972 → 1,268 토큰** (개편 전 · 직후 · 현재, 같은 토큰나이저로 잼 값)

근거 Andrej Karpathy, *LLM Wiki: A Pattern for Persistent Knowledge Bases*

도구 Obsidian 볼트로 열림 · 생성·검사 스크립트 7종

- 1 LLM 위키란 무엇인가 — 처음 듣는 사람을 위해
- 2 왜 도입했나 — 2026년 7월 31일의 상태
- 3 지금의 구조 — 세 개의 층
- 4 그래프로 본 위키 — 옵시디언으로 무엇을 관찰하나
- 5 무엇이 달라졌나 — 숫자
- 6 팀 협업에서의 쓸모
- 7 어떻게 쓰나 — 세 가지 연산
- 8 4주 뒤 — 구조가 견뎌나 (못 지킨 것 포함)
- 9 되돌리는 기준과 남은 비용

1. LLM 위키란 무엇인가

사람이 읽는 위키와 목적이 다르다. 독자가 에이전트다.

출발점: LLM 은 매번 처음부터 시작한다

사람과 달리 LLM 에이전트는 어제 한 일을 기억하지 못한다. 새 대화를 열면 **이 프로젝트가 무엇인지, 왜 그렇게 정했는지를 다시 알려줘야 한다**. 그 "다시 알려주는 비용"이 이 문서가 다루는 전부다.

흔히 쓰는 대응이 둘 있고, 둘 다 값을 치른다.

대응	어떻게 되나
① 큰 문서 하나를 매번 읽힌다	"우리 프로젝트 문서" 한 파일을 통째로 읽힌다. 확실하지만 질문과 무관한 내용까지 전부 값을 낸다 . 문서가 자랄수록 매 세션의 고정비가 커진다
② 대화 세션을 길게 유지한다	한 번 설명한 맥락을 잃지 않으려고 세션을 안 끊는다. 그런데 대화가 길어질수록 이후의 모든 요청이 그 누적 대화를 다시 읽는다 . 작은 수정 하나에도 앞선 맥락 전체의 값을 낸다

LLM 위키의 답

지식을 작은 노드로 쪼개고, 링크로 잇고, 필요한 것만 골라 읽게 한다. 핵심은 파일을 잘게 나누는 것이 아니라 **경계를 의미와 일치시키는 것**이다 — 노드 하나가 "결정 하나"이면, 그 결정이 궁금할 때 딱 그만큼만 읽으면 된다.

원칙	이 저장소에서의 모습
----	-------------

원본은 고치지 않는다	<code>raw/</code> — 회의록·구 서비스 분석·외부 스펙. 나중에 생각이 바뀌어도 원본은 그대로 두고 새 결정을 따로 적는다
파생 문서는 재생성한다	색인·표 페이지는 스크립트가 만든다. 손으로 고치지 않는다 — 고쳐도 다음 생성 때 날아간다
노드 = 의미 단위	결정 하나 = 파일 하나. 제목·날짜·갈래·상태를 frontmatter 에 단다
링크로 잇는다	<code>[[위키링크]]</code> . 뒤집힌 결정은 <code>superseded_by</code> 로 후속 결정을 가리킨다
찾는 규율이 이득을 만든다	<code>grep</code> 으로 파일명만 받고, 이름을 보고 골라, 그 노드만 읽는다. 매칭된 것을 전부 읽으면 쪼개기 전보다 나빠진다(5장)

사람 위키와 결정적으로 다른 점

사람은 긴 문서를 **훑어보고 필요한 곳만 눈으로 건너뛴다**. 비용이 거의 안 든다. 에이전트는 읽은 만큼 값을 낸다. 그래서 **"찾기 쉬움"이 곧 "쌈"**이고, 그것이 문서를 노드로 쪼갤 이유가 된다.

또 하나 — 사람 위키는 사람이 낚음을 눈치챈다. 여기서는 **코드도 진실이고 코드는 매 커밋 바뀐다**. 문서가 조용히 낚는다. 그래서 **lint 연산**이 따로 있다(7장).

2. 왜 도입했나 — 2026년 7월 31일의 상태

문서가 늘면서, 한 번 볼 때마다 전부를 읽어야 하는 구조가 됐다.

무엇이	왜 문제였나
DECISIONS.md 33,792 토큰	결정 56개가 한 파일에 시간순으로 쌓였다. 결정 하나는 중앙값 680토큰인데 실제로는 파일을 통째로 읽었다 — 작업 중 시스템이 "내용이 너무 커서 포함할 수 없다"며 잘라낼 정도였다
CLAUDE.md 13,518 토큰	에이전트가 매 세션 무조건 읽는 파일. BigQuery 설정처럼 특정 작업에만 필요한 내용이 항상 따라왔다
뒤집힌 결정이 그대로 살아 있었다	힌트 요금을 순차 4단계에서 선택형으로 바꿨는데 옛 결정이 같은 파일에 남아 있었다. 제목만 보면 서로 모순되는 항목도 있었다
표 하나를 알려면 네 군대를 열어야	<code>heart_transaction</code> 은 DDL 주석 · 결정 6개 · 정합성 검사 5종 · 정책 7파일에 흩어져 있었다
손으로 만든 목록이 조용히 낡았다	"비어 있는 테이블" 목록이 실제와 어긋나 있었다 — 20행짜리 표를 비었다고 적어두고, 정작 빈 표는 빠져 있었다

그리고 여섯 번째 — 세션을 끊지 못했다

이것이 문서만 봐서는 안 보이는, 실제 작업 방식의 문제였다.

맥락을 다시 설명하는 것이 비싸니까 세션을 일부러 유지했다. 스키마 컬럼 하나를 고치는 작은 일도, 새 대화를 열어 "이 프로젝트는 이런 거고 저 표는 이래서 이렇게 만들었고..."를 다시 말하느니 **어제 쓰던 대화창을 이어 쓰는 편이 싸 보였기 때문이다.**

그런데 그 선택은 정반대의 값을 치른다. 대화가 길어질수록 그 뒤의 모든 요청이 누적된 대화 전체를 다시 실는다. 작은 수정 하나가 앞선 두 시간치 맥락의 값을 낸다. 게다가 길어진 세션에는 이미 끝난 작업의 시행착오·실패한 시도·중간 출력이 그대로 남아 있어서, **정작 필요한 정보의 밀도는 계속 떨어진다.**

진짜 문제는 토큰이 아니라 의존이었다

세션에만 있는 맥락은 **저장소에 없는 맥락**이다. 그 대화창을 닫는 순간 사라지고, 팀원은 처음부터 볼 수 없으며, 몇 주 뒤의 나도 못 찾는다. "왜 그렇게 정했나"가 **대화 기록에만 있는 상태였다.**

위키로 바꾼 뒤에는 세션을 짧게 끊을 수 있게 됐다. 새 대화를 열고 필요한 노드 하나(평균 1,268 토큰)만 읽으면 그 작업에 필요한 맥락이 선다. 작은 작업일수록 이득이 크다 — 작은 작업에 긴 세션을 물고 있는 것이 가장 비쌌기 때문이다.

다섯 문제의 공통점

전부 **오류를 내지 않는다**. 문서는 잘 읽히고 링크도 멀쩡하다. 다만 낡았거나, 필요 없는 것까지 딸려올 뿐이다. 이 프로젝트가 스키마에서 계속 상대해 온 **"조용히 틀리는 것"**이 문서에서도 똑같이 나타난 것이다.

3. 지금의 구조 — 세 개의 층

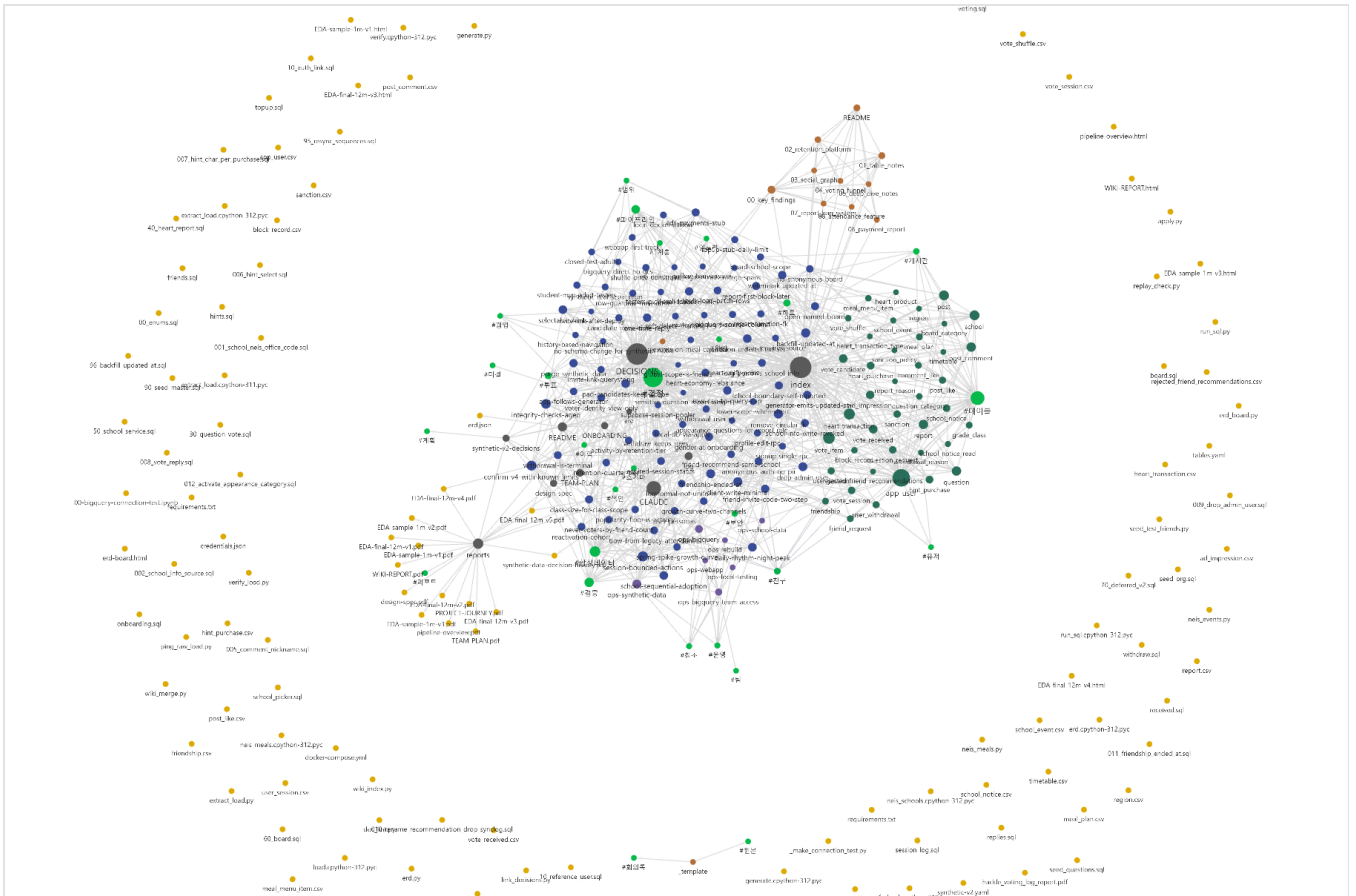
원본은 사람만 고치고, 위키는 에이전트가 소유하고, 규약이 그 일을 정의한다.

1층 · 원본 raw/ 회의록 · 구 서비스 분석 · 외부 스펙 사람만 고친다. 에이전트는 읽기만 9개 노드	2층 · 위키 docs/ 원본과 코드에서 뽑아 만든 것 에이전트가 소유한다 결정 91 · 표 40 · 운영 10 · 색인	3층 · 규약 CLAUDE.md 그 일을 어떻게 하는지 사람과 에이전트가 함께 다듬는다 매 세션 자동으로 읽힌다
--	--	---

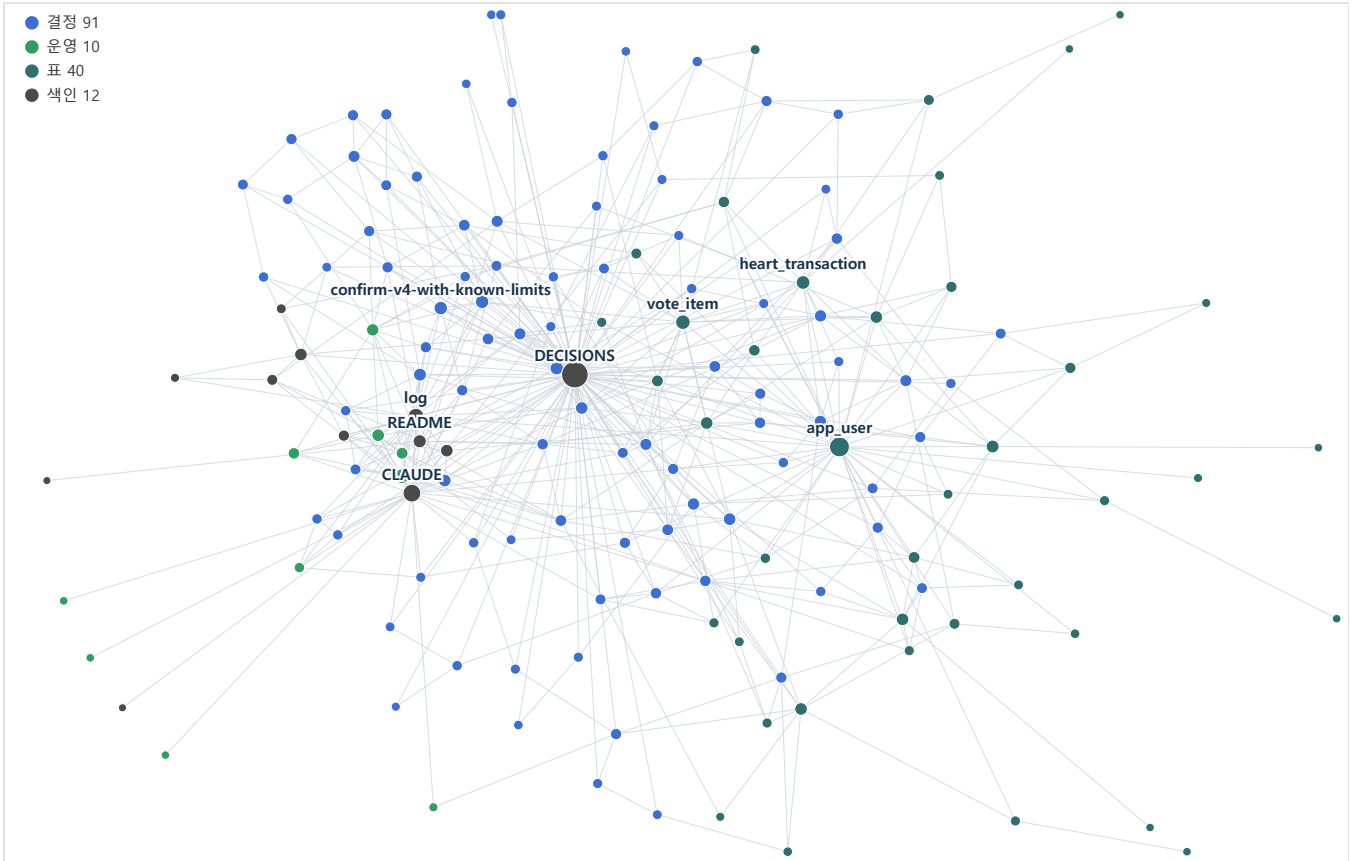
갈래	개수	무엇이 들어 있나
결정 docs/decisions/	91	결정 하나 = 파일 하나. 결정 / 이유 / 대안 / 영향. 갈래는 검증 21 · 파이프라인 18 · 투표 8 · 보안 7 · 하트 6 순. 뒤집힌 것 4건 은 취소선과 화살표로 후속 결정을 가리킨다
표 docs/tables/	40	생성물 . 표 하나의 DDL·결정·검사·정책을 한 장에 모은다
운영 docs/ops/	10	"이 작업 어떻게 하지"에 답하는 절차 문서
색인·규약	13	index (생성물) · DECISIONS · CLAUDE · log · README · TEAM-PLAN · ONBOARDING 등
원본 raw/	9	불변. 바뀐 것은 새 결정 노드로 적는다

4. 그래프로 본 위키

노드와 링크를 그림으로 보면 문서로는 안 보이는 것이 보인다.



옵시디언 그래프 뷰에서 본 저장소 전체. 가운데 촘촘한 덩어리가 위키다 — 결정(파랑) · 표(청록) · 운영(초록) · 색인(회색)이 서로 이어져 있고, DECISIONS · index · CLAUDE 가 큰 원으로 보인다. 바깥에 흩어진 노란 점들은 위키가 아니다 — SQL · 스크립트 · 데이터 파일이라 링크가 없다. 이 대비가 곧 "무엇이 문서화돼 있고 무엇이 아닌가"다.



같은 데이터를 재현 가능한 형태로 다시 그린 것 — `python db/wiki_graph.py` (노드 153 · 링크 505). 스크린샷은 찍은 순간의 것이고 배치가 매번 달라지지만, 이쪽은 씨앗이 고정돼 같은 그림이 다시 나온다. 위키 노드만 남기고 SQL·스크립트는 뺐으며, 생성 색인(`index`)도 뺐다 — 모든 노드를 가리켜서 그리면 그래프가 통째로 별 모양이 되고 결정끼리의 구조가 안 보인다.

이 그림에서 읽는 것

보이는 것	뜻
가운데 큰 회색 — <code>DECISIONS</code>	결정 색인. 모든 결정이 여기로 모인다
오른쪽 청록 뭉치 — <code>app_user</code> · <code>heart_transaction</code> · <code>vote_item</code>	표 페이지들. 결정이 많이 얹힌 표일수록 크다. 이 표를 건드리면 무엇이 달려 오는지가 그대로 보인다
파란 점들의 그물	결정 91개. 서로 이어져 있으면 <code>link_decisions.py</code> 가 '이어지는 결정'을 붙인 것이다
바깥의 외톨이들	링크가 하나뿐인 노드. 나쁜 신호는 아니지만 볼 값은 있다 — 아무도 안 가리키는 결정은 잊힌 결정이 되기 쉽다

옵시디언으로 실제로 잡은 것 셋

이 저장소의 `docs/` 는 옵시디언 볼트로 그대로 열린다. 링크가 그래프가 되고 frontmatter 가 속성이 된다. 별도 도구를 붙이지 않았다.

#	그래프를 보다가 발견한 것
1	결정 56개가 통째로 사라져 있었다. 루트 색인 하나만 그래프에서 빠려고 <code>-path:"DECISIONS"</code> 필터를 걸었는데, 옵시디언의 <code>path:</code> 는 대소문자를 구분하지 않아 <code>docs/decisions/</code> 전체가 걸려졌다. 그래프가 텅 비어서 알았다 — 파일 목록으로는 못 봤을 사고다
2	별 모양이 되는 것을 막았다. 모든 노드가 색인만 가리키면 그래프가 바퀴살이 된다. 그래서 참조는 되도록 개별 노드를 가리킨다는 규칙을 규약에 박고, <code>link_decisions.py</code> 로 결정끼리 잇게 했다

3	색인에서 빠진 노드가 보였다. group: 웹앱 인 노드 3개가 그 갈래 자체가 색인에 없어서 통째로 빠져 있었다. 파일은 멀쩡한데 아무도 못 찾는 상태다 — 색인에 없는 노드는 없는 노드다. 이걸 결국 lint 검사로 옮겼다
---	--

그림은 문서가 아니라 구조를 보여준다

세 사고 모두 "문서를 읽어서" 발견한 것이 아니다. 모양이 이상해서 발견했다. 한 파일 시절에는 볼 모양 자체가 없었다.

5. 무엇이 달라졌나 — 숫자

토큰 절감이 가장 크고, 그다음이 낡음 탐지다.

세 단계로 한눈에

같은 토큰나이저로 **개편 전** · **개편 직후** · **현재** 세 시점을 잴다. 옛 상태는 git 에서 꺼냈다(`pre-wiki` · `9a10db0`).

결정 하나를 알기 위해 읽는 토큰

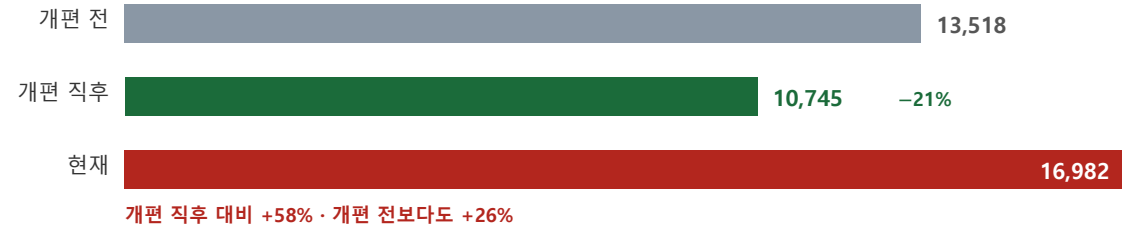


개편 전은 실제로 파일을 통째로 읽었다. 아래는 그 작은 값들만 확대한 것이다.



위 세 시점 — 개편 전 막대는 **그때 실제로 하던 습관**(통째로 읽기)이지 한 파일의 최선이 아니다. 한 파일의 최선은 3,206 이고 그것과 비교해도 D 가 2.5배 싸다("이 비교가 공정한가"). **아래** 작은 값 확대. 회색은 **지금 규모를 한 파일로 되돌렸을 때** grep 으로 구간만 읽는 비용(4,484)이다 — 공정한 기준선은 이쪽이고, 그에 비해서도 71.7% 싸다.

매 세션 고정비 — **CLAUDE.md** (모든 세션이 무조건 지불한다)



개편 때 덜어낸 것을 4주 만에 도로 채웠다. 함정을 겪을 때마다 규약이 한 줄씩 늘기 때문이다. 이 보고서에서 유일하게 나빠진 숫자다(8장).

단계	결정 하나 찾기	매 세션 고정비	그때 실제로 하던 방식
개편 전 7/30	33,792	13,518	결정 56개짜리 한 파일을 통째로 읽었다
개편 직후 7/31	972	10,745	grep 으로 파일명만 받고 노드 하나만
현재 8/25	1,268	16,982	같은 방식. 결정은 91개로 늘었다

두 숫자가 반대 방향으로 갔다

질문당 비용은 33,792 → 1,268 로 줄었고 위키가 커져도 거의 안 늘었다. 세션당 고정비는 13,518 → 16,982 로 늘었다. 전자는 구조가 만든 이득이고, 후자는 규율이 무너져 생긴 손해다. **지금 이 위키에서 가장 값싼 개선은 규약을 다시 덜어내는 것이다.**

경로별로 나눠 보기

질문 다섯(하트 경제 · 친구 끊기 · 합성 데이터 · 증분 합정 · 힌트 요금)으로 경로별 비용을 잰다. **2026-08-25** 에 다시 잰고, 비교를 위해 **개편 직후** 상태를 **git** 에서 꺼내 같은 토큰나이저로 함께 잰다.

경로	노드 56개 (2026-07-31)	노드 91개 (2026-08-25)	무엇인가
A 통째로 읽기	34,796	92,384	쪼개기 전에 실제로 하던 것
B grep + 구간 읽기	2,535	4,484	쪼개기 전에도 가능했던 것 — 공정한 기준선
D grep + 정확한 노드	972	1,268	쪼갠 뒤 하게 되는 것
E grep + 매칭 전부	8,822	31,655	규율이 무너졌을 때
B 대비 D 절감	61.6%	71.7%	

2.6배

위키 전체 크기 증가

+30%

골라 읽기 비용 증가

71.7%

절감 (61.6% →)

7.1배

규율을 어겼을 때(E/B)

위키가 커질수록 이득도 커졌다. 전체는 2.6배가 됐는데 골라 읽기는 30%만 늘었다 — 노드 하나의 크기는 위키 크기와 무관하기 때문이다. 읽기 창을 $\pm 15/\pm 40$ 줄로 바뀌도 방향은 같다(52→68%, 69→75%).

규율을 어겼을 때의 벌금은 두 배가 됐다

매칭된 노드를 전부 읽으면 한 파일 시절의 **7.1배**다(56개 시절엔 3.5배). "합성"이라는 낱말은 이제 **91개 중 47개**에 걸리고, 그것을 다 읽으면 58,940 토큰이다. 구조가 아니라 **규율이 이득을 만든다**는 말이 4주 만에 더 사실이 됐다.

이 비교가 공정한가

당연히 나올 반론이 있다. "한 파일이어도 전부 읽지는 않는다. 통째로 읽기(A)와 골라 읽기(D)를 견주면 **사기 아닌가?**" 맞는 말이고, 이 프로젝트가 실제로 한 번 그렇게 틀렸다 — 처음엔 A 를 기준으로 **"91% 절감"**이라 적었다가 **"그건 게으른 선택이었지 유일한 방법이 아니었다"**며 스스로 취소했다. 그래서 기준선이 B 다.

그러면 남은 질문은 **"B 를 얼마나 잘 잡았느냐"**다. 읽기 창을 줄여 가며 재봤다 — 창이 0 이면 **본문을 한 줄도 안 읽는다는** 뜻이다.

읽기 창	B 합계	└ grep 출력	└ 본문 읽기	D	절감
± 0 줄	3,206	3,180	26	1,268	60.5%
± 15 줄	3,930	3,180	750	1,268	67.7%
± 25 줄	4,484	3,180	1,304	1,268	71.7%
± 40 줄	5,131	3,180	1,951	1,268	75.3%

한 파일 경로의 비용은 본문 읽기가 아니라 검색 결과 자체다

창을 0 으로 줄여 아무것도 읽지 않아도 3,206 토큰이고, 그래도 D 의 2.5배다. 이유는 구조적이다 — 한 파일에서 grep 하면 결과가 '줄'이고, 노드로 쪼개면 '파일 이름'이다.

질문	한 파일 grep 출력	노드 파일명 목록	D 합계
친구 끊기	4,843	27개 파일명	1,450
합성 데이터	3,753	47개 파일명	1,362
하트 경제	3,570	21개 파일명	939
힌트 요금	2,301	20개 파일명	1,515
증분 합정	1,433	13개 파일명	1,073

"하트" 같은 낱말은 한 파일에서 수십~수백 줄에 걸리고 그 줄들은 전부 본문이라 길다. 파일명은 한 줄에 10토큰 남짓이고 파일당 하나뿐이다. 같은 낱말로 검색해도 돌아오는 것의 성격이 다르다.

반대로, D 에도 유리한 가정이 하나 있다

파일명만 보고 한 번에 맞는 노드를 고른다고 가정한 값이다. 잘못 고르면 다른 노드를 하나 더 읽어야 하고(+약 1,000), 두 번 틀리면 B 와 비슷해진다. 이 경로의 이득은 이름을 보고 고르는 판단이 맞을 때 만 성립한다 — 그래서 결정 노드의 제목과 파일명을 짓는 일이 실제 작업이다.

그리고 3단계 그림의 빨간 막대(33,792)는 A 다. 그것은 "그때 실제로 하던 습관"이지 "한 파일의 최선"이 아니다. 한 파일의 최선은 위 표의 3,206 이고, 그와 비교해도 D 가 2.5배 싸다.

세션을 끊을 수 있게 됐다

2장의 여섯 번째 문제에 대한 답이다. 지금은 작은 작업마다 새 세션을 열고, 필요한 노드만 읽어 시작한다.

작업 방식	맥락을 세우는 비용	남는 것
세션을 계속 이어 쓰기	0 처럼 보이지만, 이후 모든 요청이 누적 대화를 다시 싣는다	대화창을 닫으면 사라진다
새 세션 + 노드 골라 읽기	1,268 토큰 (결정 하나 기준)	저장소에 남는다. 팀원도 본다

그 밖의 절감

작업	전	후
표 하나 파악	4개 파일을 열어 조합	표 페이지 1장 (484 토큰)
뒤집힌 결정 찾기	제목만 보면 알 수 없음	색인에서 취소선 + 후속 결정 화살표
낡은 서술 찾기	사람이 눈으로	doc_lint.py 9종 검사

6. 팀 협업에서의 쓸모

에이전트를 위해 만든 구조인데, 사람에게 먼저 쓰였다.

팀원 4명이 붙어 있다(BigQuery 관리자 권한). 위키가 협업에서 실제로 한 일은 셋이다.

무엇	어떻게 쓰이나
합류 경로가 문서 하나	ONBOARDING 이 계정 없이 되는 A 갈래(로컬 합성 데이터)와 계정이 필요한 B 갈래를 나눠 적는다. 새로 온 사람에게 이것만 가리키면 된다
영향 범위가 한 장에	스키마를 바꿀 때 docs/tables/<표>.md 한 장이 그 표에 걸린 결정·검사·정책을 다 보여준다. "이거 고치면 뭐가 깨지죠"에 답하는 문서다. 절차는 TEAM-PLAN 에 있다
결정이 리뷰 단위가 된다	결정 하나가 파일 하나라 커밋 diff 에 그 결정만 뜬다. 한 파일 시절에는 긴 문서의 어느 줄이 바뀌었는지 봐야 했다
도구를 강요하지 않는다	마크다운이라 에디터·깃허브에서 그냥 읽히고, 옵시디언으로 열면 그래프와 백링크가 덩으로 온다. 팀원에게 새 도구를 깔라고 하지 않아도 된다

정직하게 — 협업 기능 중 하나는 4주 동안 놀았다

회의록 ingest 는 한 번도 쓰이지 않았다. raw/meetings/ 에는 템플릿만 있다. 회의가 없었기 때문이고, 결정 35건이 전부 "작업하다 판단이 서면 그 자리에서" 경로로 났다. 구조를 만들었다고 그 경로가 쓰이는 것은 아니다.

7. 어떻게 쓰나 — 세 가지 연산

연산	언제	무엇을 하나
ingest	원본을 넣었을 때	무엇이 결정이고 무엇이 논의인지 가른다 → 결정 노드를 만들고 → 영향받는 문서를 고치고 → 생성물을 다시 뽑고 → 이력에 한 줄 남긴다
query	물어볼 때	grep 으로 파일명만 받고, 이름을 보고 골라, 그 노드만 읽는다. 색인까지 읽을 필요도 보통 없다. 답이 길고 다시 쓰일 것 같으면 그 답 자체를 새 노드로 남긴다
lint	문서·스키마를 손댄 뒤	문서의 주장을 살아 있는 DB·파일시스템과 대조한다. 9종

`doc_lint.py` 가 보는 아홉 가지 — 숫자 · 사라진 이름 · 끊어진 링크 · 없는 스크립트 · 낡은 생성물 · 미반영 회의록 · 없는 질문 번호 · 결정 없이 쌓인 코드 · 색인에서 빠진 결정 노드. 뒤의 셋은 처음엔 없었다 — 전부 실제로 당하고 나서 붙였다.

★ 는 고쳐야 하고, · 는 사람이 판단한다

숫자와 이름은 이력 서술과 기계적으로 구별되지 않는다. "그때는 30개였다"는 틀린 게 아니다. 그래서 알리기만 한다. 전부 실패로 두면 늘 빨간불이 되고, 그러면 검사 자체를 안 보게 된다.

8. 4주 뒤 — 구조가 견뎠나

결정이 56 → 91 개가 됐다. 늘어난 35건이 구조를 시험했다.

항목	결과
끊어진 링크 0	지켜졌다. 노드가 63% 늘었는데도 0 이다 — lint 가 실패로 잡기 때문이지 조심해서가 아니다
토큰 이득	커졌다. 61.6% → 71.7% (5장)
뒤집힌 결정이 드러난다	4건. 한 파일 시절이었다면 옛 결정이 살아남아 누군가 그것을 근거로 삼았을 것이다
결정을 노드로 적기	⚠ 아홉 번 어겼다. 그래서 검사를 붙였다 — 사람의 규율에 기대는 항목은 결국 어긋난다
회의록 ingest	⚠ 0회. 회의가 없었다
CLAUDE.md 를 얇게	❌ 못 지켰다. 개편 때 13,518 → 10,745 로 줄였는데 지금 16,982 토큰이다 — 개편 전보다도 26% 크다

가장 크게 실패한 항목 — 매 세션 고정비가 두 배가 됐다

CLAUDE.md 는 모든 세션이 무조건 지불하는 비용이라 위키에서 제일 비싼 파일이다. 함정을 겪을 때마다 한 줄씩 늘어난 결과, 개편 때 덜어낸 것을 도로 채우고도 개편 전보다 커졌다. **노드를 쪼개 아낀 것보다 이쪽에서 잃은 것이 클 수 있다** — 골라 읽기가 질문당 1,268 토큰인데, 규약은 세션마다 16,982 토큰이다.

중복 117줄을 덜어내는 작업이 대기 중이다. 이것이 지금 이 위키의 가장 큰 미해결 항목이다.

9. 되돌리는 기준과 남은 비용

판단 기준은 둘이다 — 같은 질문에 **토큰을 더 쓰거나**, 관련 결정을 놓쳐 **답이 나빠지거나**. 4주 뒤 기준으로 둘 다 발생하지 않았다. 그래도 되돌리는 스크립트(`db/wiki_merge.py`)는 유지한다 — 지금 있는 노드 전부를 읽어 합치므로 개편 이후에 쓴 결정도 살아남는다. `git revert` 로 하면 그것들이 함께 사라진다.

남은 비용 — 정직하게

무엇	지금 값
규약 파일이 매 세션 붙는다	16,982 토큰. 개편 직후 대비 +58%. 최대 미해결 항목
색인 자체가 무겁다	10,545 토큰. 전체 탐색 때는 이 값을 낸다(보통은 안 읽는다)
규율이 무너지면 더 나쁘다	매칭 전부 읽기 = 한 파일 시절의 7.1배
파일 목록이 커지고 있다	골라 읽기 비용 중 파일 목록 비중이 13% → 24%. 노드 180~200개 근처에서 다시 볼 값이다
검사를 늘리는 일은 안 끝난다	lint 6종 → 9종. 셋 다 당하고 나서 붙였다

재현

```
# 토큰 실측 - 두 시점을 같은 자로 켜다
python db/token_bench.py --window 15,25,40

# 이 보고서의 그래프 그림
python db/wiki_graph.py

# 문서가 실제와 어긋났는지
python db/doc_lint.py

# 되돌리기 (노드 → 한 파일)
python db/wiki_merge.py --dry-run
```